Тема 1. Введение в структуры и алгоритмы обработки данных

Лекция 1

Введение в теорию алгоритмов. Блок-схема алгоритма

Историческая справка

Содержательные явления, приведшие к возникновению понятия «алгоритм», прослеживаются в математике в течение всего его времени существования. Со школьной скамьи молодые люди изучают алгоритм Евклида нахождения наибольшего общего кратного натуральных чисел, найденный еще в III веке до нашей эры и доживший до наших дней. В XV веке был известен алгоритм, разработанный самаркандским астрономом Аль-Каши, вычисления числа π, которое он вычислил с 17 верными значащими цифрами после запятой. Список «старинных» алгоритмов можно продолжать и далее, однако само понятие алгоритма сформировалось как предмет самостоятельного изучения лишь в XX веке.

Первоначально понятие алгоритма рассматривалось в общей форме в виде словесных правил, схем, формул (ныне называемые методиками расчета) и использовались для описания вычислительного процесса. Они не являлись точным математическим определением, а лишь объясняли смысл слова «алгоритм». С алгоритмами, т.е. эффективными процедурами, однозначно приводящими к результату, математики имели дело всегда. Школьные методы умножения «столбиком» и деления «углом», метод исключений неизвестных при решении систем линейных уравнений – все это примеры алгоритмов. На протяжении длительного времени понятие алгоритм в своей основе не изменялось, приобретая все большую выразительность. Традиции организации вычислений складывались веками и стали составной частью общей научной культурой. Все многообразие вычислений комбинировалось из 10 – 15 четко определенных операций арифметики, тригонометрии и анализа. Поэтому понятие метода вычислений считалось изначально ясным, не требующих специальных исследований.

Начиная с 20-х 30-х годов XX столетия в связи с вопросами обоснования математики, развитием вычислительной математики и вычислительной техники возникла необходимость уточнения понятия алгоритм как объекта математической теории. Появился раздел дискретной математики называемый *теорией алгоритмов*. Неслучайно, что основоположниками теории алгоритмов являются великие математики XX века такие, как А.И. Колмогоров, А.А. Марков, А.П. Ершов, А.И. Мальцев, В.А. Успенский, А.М. Тьюринг, К. Гёдель, А. Чёрчь, А. Туэ, Э.Л. Пост, С.К. Клини и др.

Толчком к развитию теории алгоритмов послужили исследования А. Чёрча (1936 г.) связанные попыткой понять, что же мы можем вычислить с помощью функций? Чёрч уточнил понятие *вычислимой функции* и привел пример функции не являющейся вычислимой. Поскольку *вычислимая функция* определяется как функция, для которой существует алгоритм её вычисляющей, то естественно потребовались исследования, уточняющего понятие алгоритма.

В настоящее время исследования в области теории алгоритмов развиваются в следующих направлениях:

1. исследование классов вычислимых функций (изучение класса *рекурсивных и перечислимых множеств*, сравнение и *классификация алгоритмической природы произвольных подмножеств множества натуральных чисел*, *теория нумераций* и т.д.);

2. исследование механизмов вычисления (*теория конечных автоматов* как модели вычислительного механизма, *растущие автоматы* – самовоспроизводящиеся машины, *машины с оракулом* – развитие понятия алгоритма);

3. изучение понятия сложности алгоритма (разработка методов оценки *сложности алгоритмов и вычисления*).

Теория алгоритмов имеет множество приложений в математической логике и теории моделей, образует теоретический фундамент для ряда вопросов вычислительной математики и тесно связана с кибернетикой и информационными технологиями.

Этапы решения задач на ЭВМ

Процесс создания компьютерной программы для решения какой-либо практической задачи состоит из нескольких этапов: постановка; формализация; создание технического задания на исходную задачу; разработка алгоритма решения задачи; кодирование, тестирование, отладка и документирование программы; получение решения исходной задачи путем выполнения законченной программы. Рассмотрим эти этапы подробнее.

Постановка задачи заключается в определении существенно важных объектов и (или) явлений предметной области, определении отношений между этими объектами и (или) явлениями, выделении и описании объекта исследования, исходных данных и результатов решения задачи.

В первом приближении большинство задач, встречающихся на практике, не имеют четкого и однозначного описания. Определенные задачи, такие как разработка рецепта вечной молодости или сохранение мира во всем мире, вообще невозможно сформулировать в терминах, допускающих компьютерное решение. Если мы предполагаем, что наша задача может быть решена на компьютере, обычно даже для неформального ее описания требуется огромное количество разнообразных параметров. И часто только в ходе дополнительных экспериментов можно найти интервалы изменения этих параметров.

Таким образом, если задача поставлена в общем виде, не сформулирована точно, то необходимо понять сущность задачи и правильно ее сформулировать.

Формализация заключается в разработке формальной (математической) модели исследуемого объекта или явления. Математическая модель - это описание некоторого класса объектов или явлений внешнего мира, выраженное с помощью математической символики. То есть, разработка математической модели буквально означает, что неформальное описание конкретного объекта или явления переводится на язык математических формул и дальше рассматривается как математическая проблема.

Таким образом, если определенные аспекты решаемой задачи можно выразить в терминах какой-либо математической модели, то это, безусловно,

необходимо сделать, так как в этом случае в рамках такой формальной модели мы можем узнать, существуют ли методы и алгоритмы решения нашей задачи. Даже если такие методы и алгоритмы не существуют на сегодняшний день, то привлечение средств и свойств формальной модели поможет в построении "подходящего" решения исходной задачи.

Практически любую область математики или других наук можно привлечь к построению модели определенного круга задач. Для задач, числовых по своей природе, можно построить модели на основе общих математических конструкций, таких как системы линейных уравнений (например, для задач расчета электрических цепей или напряжений в закрепленных балках), дифференциальные уравнения (задачи прогноза роста популяций или расчета скорости протекания химических реакций). Для задач с символьными или текстовыми данными можно применить модели символьных последовательностей или формальных грамматик. Решение таких задач содержит этапы компиляции (преобразование программ, написанных на языке высокого уровня, в программы на машинно-ориентированных языках) и информационного поиска (распознавание определенных слов в списках заголовков каких-либо библиотек и т.п.).

Создание технического задания на исходную задачу заключается в уточнении исходных данных, результата и способа получения результата из исходных данных, проводимого в рамках полученной математической модели. Учитывая специфику решения задачи – с применением средств вычислительной техники, способ получения результата из исходных данных должен быть представлен в форме алгоритма, состоящего из конечной последовательности инструкций, каждая из которых имеет четкий смысл и может быть выполнена с конечными вычислительными затратами за конечное время. Т.е. на этом этапе должны быть установлены выходные и входные данные алгоритма, выделены основные отношения между ними. Необходимо определить, может ли задача быть решена и при каких исходных данных. Результат этапа – спецификация алгоритма, то есть формулировка в общем виде того, что должен делать алгоритм, чтобы переработать входные данные в выходные. Причем должен быть описан класс входных данных со всеми ограничениями.

Разработка алгоритма решения задачи и его запись заключается в создании нескольких формализованных описаний алгоритма на нескольких уровнях абстракции. Этот процесс (его можно назвать пошаговой кристаллизацией) начинается от практически неформального описания алгоритма, выполняемого в терминах применяемой математической модели и содержащего обобщенные и укрупненные операторы, и заканчивается полностью формальным описанием, в котором каждая инструкция имеет четкое определение.

Следует отметить, что при разработке алгоритма, как последовательности действий по преобразованию информации, приходится разрабатывать и описания информационных объектов, над которыми эти преобразования и производятся. При этом в зависимости от уровня абстракции алгоритма, описания информационных объектов так же могут иметь различную степень формальности – начиная от совершенно абстрактного и заканчивая полностью формальным описанием, выполненным в терминах языка программирования.

Кодирование, тестирование, отладка и документирование программы. Кодирование - это заключительный этап пошаговой кристаллизации алгоритма – процесс записи алгоритма на выбранном языке программирования. Его результатом является компьютерная программа, решающая поставленную задачу, но возможно содержащая ошибки. ***Тестирование и отладка*** – это итеративный процесс установления факта наличия ошибок в получившейся программе, их локализации и устранении. ***Документирование*** заключается в создании описания способа применения программы для решения соответствующей задачи. Такое описание должно содержать четкие инструкции пользователю о том, как необходимо запускать программу, как подготавливать и вводить в нее исходные данные, как получать и интерпретировать результаты ее работы.

Получение и анализ результатов – последний этап решения задачи на ЭВМ, который заключается в организации запуска программы на выполнение в соответствии с пользовательской документацией.

Первые четыре из рассмотренных этапов создания программ для решения прикладных задач являются самыми важными и фактически определяют успех всей разработки.

Таким образом, создание компьютерной программы можно рассматривать как процесс последовательного преобразования информации от первоначальной неформальной постановки задачи, до получения завершенной программы на языке программирования. Причем это преобразование затрагивает как описания информационных объектов задачи (данные) так и описания действий над этими объектами (алгоритмы). Схема процесса создания программы, иллюстрирующая рассмотренные выше преобразования, приведена на рис. 1.1.

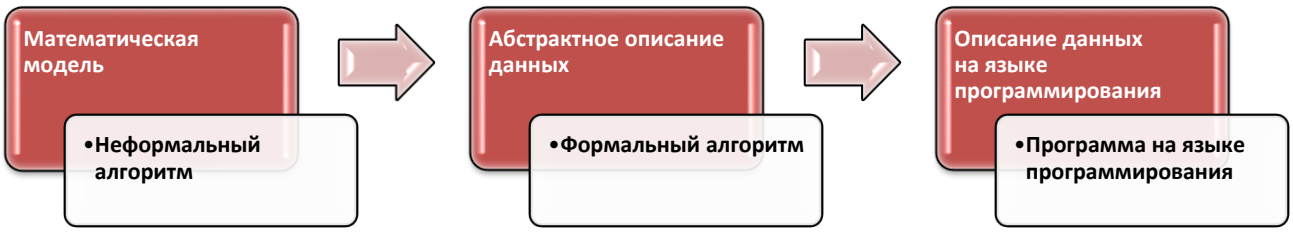


Рис. 1.1. Схема процесса создания программ для решения прикладных задач

Понятие алгоритма

Как было указано в предыдущем параграфе, в процессе преобразования информации от постановки задачи до получения компьютерной программы выделяются два взаимосвязанных объекта – данные и алгоритм их преобразования. Уточним эти понятия.

Понятие алгоритма является центральным понятием информатики. Слово «алгоритм» произошло от имени узбекского математика аль-Хорезми, который еще в IX веке сформулировал правила выполнения арифметических действий. В современной математике и информатике термин ***алгоритм*** имеет следующие определения:

- последовательность действий со строго определенными правилами выполнения;
- предписание, определяющее содержание и последовательность операций, переводящих исходные данные в искомый результат;

- точное описание некоторого вычислительного процесса или любой иной последовательности действий;
- точное и полное предписание о последовательности выполнения конечного числа действий, необходимых для решения любой задачи данного типа.

Алгоритм может быть предназначен для выполнения его человеком или автоматическим устройством - формальным исполнителем. Задача исполнителя - точная реализация уже имеющегося алгоритма. Формальный исполнитель не обязан вникать в сущность алгоритма, а возможно, и неспособен его понять.

Примером формального исполнителя может служить стиральная машина-автомат, которая неукоснительно исполняет предписанные ей действия, даже если вы забыли положить в нее порошок. Человек тоже может выступать в роли формального исполнителя, но в первую очередь формальными исполнителями являются различные автоматические устройства, и компьютер в том числе. Каждый алгоритм создается в расчете на конкретного исполнителя.

Каждый исполнитель может выполнять команды только из некоторого строго заданного списка — системы команд исполнителя. Для каждой команды должны быть заданы условия применимости (в каких состояниях среды может быть выполнена команда) и описаны результаты выполнения команды. После вызова команды исполнитель совершает соответствующее элементарное действие.

В информатике универсальным исполнителем алгоритмов является компьютер.

Виды алгоритмов

Алгоритм применительно к вычислительной машине - точное предписание, т. е. набор операций н правил их чередования, при помощи которого, начиная с некоторых исходных данных, можно решить любую задачу фиксированного типа.

Алгоритмы в зависимости от цели, начальных условий задачи, путей ее решения, определения действий исполнителя подразделяются следующим образом:

- **Вероятностный (стохастический) алгоритм** дает программу решения задачи несколькими путями или способами, приводящими к вероятному достижению результата.

- **Эвристический алгоритм** (от греческого слова «эврика») — это такой алгоритм, в котором достижение конечного результата программы действий однозначно не предопределено, так же как не обозначена вся последовательность действий, не выявлены все действия исполнителя. К эвристическим алгоритмам относят, например, инструкции и предписания. В этих алгоритмах используются универсальные логические процедуры и способы принятия решений, основанные на аналогиях, ассоциациях и прошлом опыте решения схожих задач.

- **Линейный алгоритм** — набор команд (указаний), выполняемых последовательно друг за другом.

- **Разветвляющийся алгоритм** — алгоритм, содержащий хотя бы одно условие, в результате проверки которого ЭВМ обеспечивает переход на один из двух возможных шагов.

- **Циклический алгоритм** — алгоритм, предусматривающий многократное повторение одного и того же действия (одних и тех же операций) над новыми исходными данными. К циклическим алгоритмам сводится большинство методов вычислений, перебора вариантов. *Цикл программы* — последовательность команд (серия, тело цикла), которая может выполняться многократно (для новых исходных данных) до удовлетворения некоторому условию.

- **Вспомогательный (подчиненный) алгоритм (процедура)** — алгоритм, ранее разработанный и целиком используемый при алгоритмизации конкретной задачи. В некоторых случаях при наличии одинаковых

последовательностей указаний (команд) для различных данных с целью сокращения записи также выделяют вспомогательный алгоритм.

Алгоритм можно задать несколькими способами:

- ***словесным***, то есть записью последовательности действий на естественном языке;
- ***графическим***, с помощью специальных графических символов;
- ***формульным***, то есть с помощью математических формул, которые определяют порядок вычислений;
- ***табличным***, и виде таблицы, в которой фиксируются этапы исполнения алгоритма и результаты исполнения.

Правила построения алгоритма

Чтобы алгоритм выполнил свое предназначение, его необходимо строить по определенным правилам. Поэтому нужно говорить все же не о свойствах алгоритма, а о правилах построения алгоритма, или о требованиях, предъявляемых к алгоритму.

Первое правило - при построении алгоритма, прежде всего, необходимо задать множество объектов, с которыми будет работать алгоритм. Формализованное (закодированное) представление этих объектов носит название данных. Алгоритм приступает к работе с некоторым набором данных, которые называются входными, и в результата своей работы выдает данные, которые называются выходными. Таким образом, алгоритм преобразует входные данные в выходные. Пока мы не имеем формализованных входных данных, мы не можем построить алгоритм.

Второе правило - для работы алгоритма требуется память. В памяти размещаются входные данные, с которыми алгоритм начинает работать, промежуточные данные и выходные данные, которые являются результатом работы алгоритма. Память является дискретной, т. е. состоящей из отдельных ячеек. Поименованная ячейка памяти носит название переменной. В теории

алгоритмов размеры памяти не ограничиваются, т. е. считается, что мы можем предоставить алгоритму любой необходимый для работы объем памяти.

Третье правило - дискретность. Алгоритм строится из отдельных шагов (действий, операций, команд). Точнее - из множества шагов.

Четвертое правило - детерминированность. После каждого шага необходимо указывать, какой шаг выполняется следующим, либо давать команду остановки.

Пятое правило - сходимость (результативность). Алгоритм должен завершать работу после конечного числа шагов. При этом необходимо указать, что считать результатом работы алгоритма.

Свойства алгоритма

Дискретность (прерывность, раздельность) - алгоритм должен представлять процесс решения задачи как последовательное выполнение простых (или ранее определенных) шагов. Каждое действие, предусмотренное алгоритмом, исполняется только после того, как закончилось исполнение предыдущего.

Определенность - каждое правило алгоритма должно быть четким, однозначным. Благодаря этому свойству выполнение алгоритма носит механический характер и не требует никаких дополнительных указаний или сведений о решаемой задаче.

Результативность (конечность) - алгоритм должен приводить к решению задачи за конечное число шагов.

Массовость - алгоритм решения задачи разрабатывается в общем виде, то есть он должен быть применим для некоторого класса задач, различающихся только исходными данными. При этом исходные данные могут выбираться из некоторой области, которая называется областью применимости алгоритма.

Блок-схема алгоритма

Задание алгоритмов с помощью блок-схем оказалось очень удобным средством изображения алгоритмов и получило широкое распространение.

Блок-схема алгоритма - графическое изображение алгоритма в виде связанных между собой с помощью стрелок (линий перехода) и *блоков* - графических символов, каждый из которых соответствует одному шагу алгоритма. Внутри блока дается описание соответствующего действия.

В таблице 1.1 приведены наиболее часто употребляемые символы.

Таблица 1.1

Наименование	**Обозначение**	**Функция**
Блок начало-конец (пуск-остановка)		Элемент отображает выход во внешнюю среду и вход из внешней среды (наиболее частое применение − начало и конец программы). Внутри фигуры записывается соответствующее действие.
Блок действия		Выполнение одной или нескольких операций, обработка данных любого вида (изменение значения данных, формы представления, расположения). Внутри фигуры записывают непосредственно сами операции, например, операцию присваивания: $a = 10*b + c$
Логический блок (блок условия)		Отображает решение или функцию переключательного типа с одним входом и двумя или более альтернативными выходами, из которых только один может быть выбран после вычисления условий, определенных внутри этого

		элемента. Вход в элемент обозначается линией, входящей обычно в верхнюю вершину элемента. Если выходов два или три, то обычно каждый выход обозначается линией, выходящей из оставшихся вершин (боковых и нижней). Если выходов больше трех, то их следует показывать одной линией, выходящей из вершины (чаще нижней) элемента, которая затем разветвляется. Соответствующие результаты вычислений могут записываться рядом с линиями, отображающими эти пути. *Примеры решения*: в общем случае − сравнение (три выхода: >, <, =); в программировании − условные операторы if (два выхода: true, false) и case (множество выходов)
Предопределённый процесс		Символ отображает выполнение процесса, состоящего из одной или нескольких операций, который определен в другом месте программы (в подпрограмме, модуле). Внутри символа записывается название процесса и передаваемые в него данные. Например, в программировании − вызов процедуры или функции.
Данные (ввод-вывод)		Преобразование данных в форму, пригодную для обработки (ввод) или отображения результатов обработки (вывод). Данный символ не определяет

		носителя данных (для указания типа носителя данных используются специфические символы).
Граница цикла		Символ состоит из двух частей − соответственно, начало и конец цикла − операции, выполняемые внутри цикла, размещаются между ними. Условия цикла и приращения записываются внутри символа начала или конца цикла − в зависимости от типа организации цикла. Часто для изображения на блок-схеме цикла вместо данного символа используют символ условия, указывая в нём решение, а одну из линий выхода замыкают выше в блок-схеме (перед операциями цикла).
Соединитель		Символ отображает вход в часть схемы и выход из другой части этой схемы. Используется для обрыва линии и продолжения её в другом месте (для избегания излишних пересечений или слишком длинных линий, а также, если схема состоит из нескольких страниц). Соответствующие соединительные символы должны иметь одинаковое (при том уникальное) обозначение.
Комментарий		Используется для более подробного описания шага, процесса или группы процессов. Описание помещается со стороны квадратной скобки и

		охватывается ей по всей высоте. Пунктирная линия идет к описываемому элементу, либо группе элементов (при этом группа выделяется замкнутой пунктирной линией). Также символ комментария следует использовать в тех случаях, когда объём текста, помещаемого внутри некоего символа (например, символ процесса, символ данных и др.), превышает размер самого этого символа.

Описание других элементов схем можно найти в соответствующих ГОСТ.

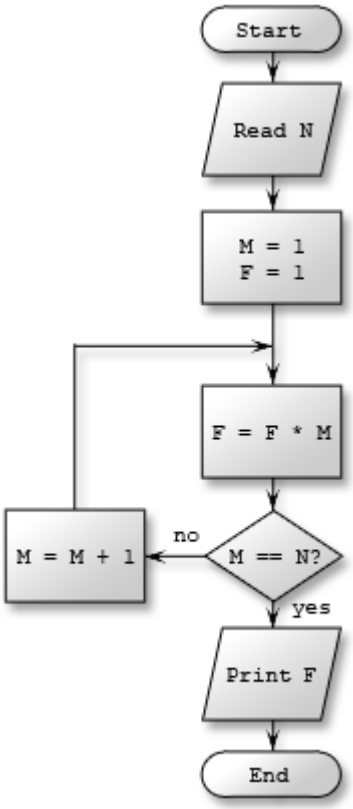


Рис. 1.1. Пример блок-схемы алгоритма вычисления факториала числа N

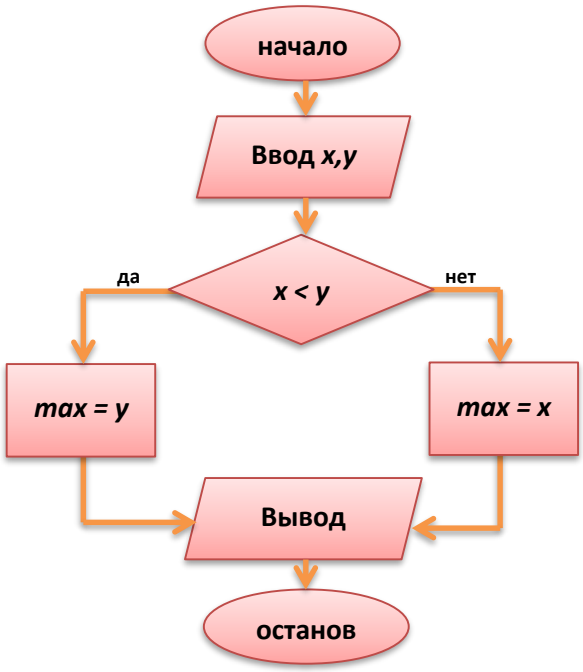


Рис. 1.2. Пример блок-схемы алгоритма нахождения максимального из двух значений

Пример. Составить блок-схему алгоритма определения высот h_a, h_b, h_c треугольника со сторонами a, b, c, если

$$h_a = \frac{2}{a}\sqrt{p\cdot(p-a)\cdot(p-b)\cdot(p-c)}, \quad h_b = \frac{2}{b}\sqrt{p\cdot(p-a)\cdot(p-b)\cdot(p-c)},$$

$$h_c = \frac{2}{c}\sqrt{p\cdot(p-a)\cdot(p-b)\cdot(p-c)}, \quad \textit{где} \quad p=(a+b+c)/2.$$

Решение. Введем обозначение

$$t = 2\sqrt{p\cdot(p-a)\cdot(p-b)\cdot(p-c)},$$

Тогда $h_a = t/a$, $h_b = t/b$, $h_c = t/c$. Блок-схема должна содержать начало, ввод a, b, c, вычисление p, t, h_a, h_b, h_c, вывод результатов и останов.

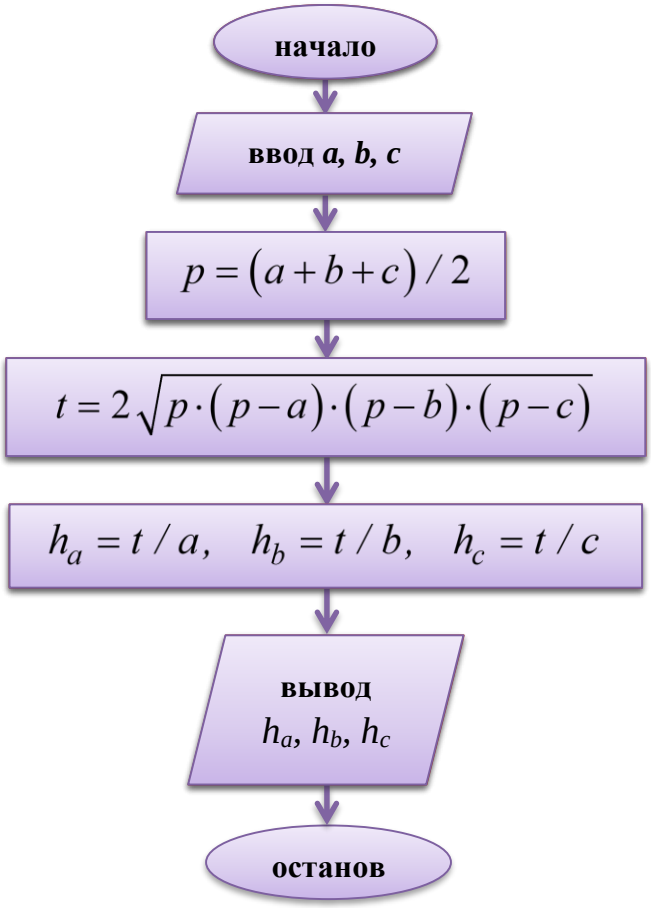


Рис.1.3. Блок-схема алгоритма определения высот треугольника

Википедия: https://ru.wikipedia.org/wiki/Блок-схема

Схе́ма - графическое представление определения, анализа или метода решения задачи, в котором используются символы для отображения данных, потока, оборудования и т. д.

Блок-схема - распространенный тип схем (*графических моделей*), описывающих алгоритмы или процессы, в которых отдельные шаги изображаются в виде блоков различной формы, соединенных между собой линиями, указывающими направление последовательности.

Стандарты выполнения

Правила выполнения схем определяются следующими документами:

- ГОСТ 19.701-90. Схемы алгоритмов, программ, данных и систем. Условные обозначения и правила выполнения.

Для программной документации (устарели, заменяются ГОСТ 19.701-90):

- ГОСТ 19.002-80. Схемы алгоритмов и программ. Правила выполнения.
- ГОСТ 19.003-80. Схемы алгоритмов и программ. Обозначения условные графические.

Данные документы в частности регулируют способы построения схем и внешний вид их элементов.

Для автоматического создания блок-схем из исходных текстов программ и их создания вручную существуют свободные программы — Dia, Kivio, OpenOffice.org Draw; коммерческие программы Microsoft Visio; также онлайн-сервисы Creately (англ.)русск.) ,Flowchart.com и LucidChart (англ.)русск.).

Распространённой и ошибочной практикой является попытка использования блок-схем для иллюстрации алгоритма на низком уровне (на уровне кода) — то есть, попытка вписывать в блоки схемы фрагменты кода на каком-либо искусственном языке. Такой подход применим только к программам, организованным согласно структурному подходу и не может отразить, к примеру, алгоритма, который реализуется во взаимодействии абстракций при объектно-ориентированном подходе.